

Independent Replication of Google Quantum AI (2022) *Suppressing quantum errors by scaling a surface code logical qubit*

arXiv:2207.06431 / Nature 614, 676–681 (2023)

Ollie (OpenClaw subagent) — QC-200 replication wave
QC-2207.06431-google-surface-code-logical-qubit

2026-07-05

Abstract

The Google Quantum AI paper reports the first superconducting-qubit demonstration in which a distance-5 rotated surface code memory *beats* the average of distance-3 subcode instances, giving a suppression factor $\Lambda_{3/5} = \varepsilon_3/\varepsilon_5 \approx 1.04 > 1$. Their headline logical error per cycle over 25 rounds is $\varepsilon_3 = (3.028 \pm 0.023)\%$ vs. $\varepsilon_5 = (2.914 \pm 0.016)\%$. We reproduce the qualitative and quantitative core of this claim on an ideal, uniform circuit-level depolarizing noise model using Stim 1.16.0 and PyMatching 2.4.0. For p well below the (circuit-noise, matching-decoder) threshold $p_{\text{th}} \sim 1\%$ we observe strong exponential suppression with distance ($\Lambda_{3/5} = 8.68$ at $p = 10^{-3}$); at $p = 10^{-2}$ we sit right at threshold ($\Lambda_{3/5} = 1.009$), which mirrors the paper’s own statement that “our device is close to threshold”; at $p = 2 \times 10^{-2}$ the trend inverts ($\Lambda_{3/5} < 1$), reproducing Fig. 4c above-threshold behaviour. **Verdict: REPLICATED** for the core headline claim $\Lambda_{3/5} > 1$ (below threshold) and for the qualitative threshold crossover; the paper’s exact percentage numbers depend on their device-calibrated noise model and are not directly tested here.

1 Paper summary

The paper implements a $d=5$ rotated surface code on a 72-qubit Sycamore- class superconducting device (25 data + 24 measure qubits) and compares its logical-memory performance to the average of four $d=3$ subgrids (9 data + 8 measure qubits per subgrid) over 25 stabilizer cycles. It is the first experimental demonstration where enlarging the code size *improves* logical performance, i.e. $\Lambda_{3/5} > 1$.

It also runs a $d=25$ repetition code and reports a logical-error floor of 1.7×10^{-6} per round dominated by a single high-energy event (1.6×10^{-7} with the event excluded), and derives an error budget mapping physical-gate error contributions to $1/\Lambda$.

2 Claims and testability

ID	Claim	Reported value	Testable in this rep?	Tested?
C1	ε_5 over 25 cycles	$(2.914 \pm 0.016)\%$	Not directly (device-cal noise)	Analog
C2	ε_3 over 25 cycles (avg 4 subgrids)	$(3.028 \pm 0.023)\%$	Not directly (device-cal noise)	Analog
C3	$\Lambda_{3/5} = \varepsilon_3/\varepsilon_5 > 1$ below threshold	≈ 1.04 (device)	Yes (Stim + MWPM)	Yes
C4	$d=25$ rep-code floor with high-E event	1.7×10^{-6} / round	No (device-specific event)	No
C5	$d=25$ rep-code floor without high-E event	1.6×10^{-7} / round	No	No
C6	Above-threshold: $\Lambda < 1$ (Fig 4c, $s = 1.3$)	qualitative	Yes	Yes

The paper’s absolute percentages (C1, C2) come from a device-calibrated, per-gate, leakage-aware, correlated-matching noise model. Reproducing them exactly would require the full experimental noise dictionary, which is outside the scope of a small-instance simulation replication. We instead test the *structural* claim C3 (that $d = 5$ beats $d = 3$ when below threshold) and the *threshold-crossover* claim C6 using the standard uniform circuit-level depolarizing model.

3 Method

1. Retrieved paper PDF from <https://arxiv.org/pdf/2207.06431> (SHA in report/artifacts_summary.md).
2. Text-extracted with `pdftotext -layout (poppler)` — see `work/paper.txt`; curated in `extraction/marker.mmd` and `extraction/nougat.mmd`.
3. Created a Python 3.14 venv and installed `stim==1.16.0` and `pymatching==2.4.0` with `pip`.
4. Built rotated-memory-Z surface-code circuits at $d \in \{3, 5, 7\}$, $r = 25$ rounds, using Stim’s built-in generator `surface_code:rotated_memory_z` with four uniform depolarizing knobs at rate p : `after_clifford_depolarization`, `after_reset_flip_probability`, `before_measure_flip_probability`, `before_round_data_depolarization`.
5. Compiled the detector error model (DEM) with error decomposition and loaded it into a PyMatching MWPM decoder (`Matching.from_detector_error_model`).
6. Sampled detector events with Stim’s compiled sampler (`compile_detector_sampler`, fixed seed per config), decoded the batch, XOR’d predictions with true observables, counted logical errors, and inferred the per-round rate via $\varepsilon = 1 - (1 - p_L)^{1/r}$.
7. Swept $p \in \{10^{-3}, 3 \times 10^{-3}, 5 \times 10^{-3}, 10^{-2}, 2 \times 10^{-2}\}$, with per- p per- d shot budgets ranging 10k–400k so the number of logical errors stays $\gtrsim 30$ except at the very lowest p /highest d corner (where 29 events still gave a clean signal).
8. Computed $\Lambda_{3/5}$ and $\Lambda_{5/7}$ per p , plotted both ε_d vs. p and Λ vs. p (`report/evidence/fig_eps_vs_p.pdf`, `fig_lambda_vs_p.pdf`).

All code + logs: `report/evidence/surface_code_sim.py`, `make_plots.py`, `sim_run.log`, `results.json`, `results.csv`, `example_circuit_d5_r25_p1e-3.stim`. Total wall-clock: 80.8 s on a 2020 M-series Mac (CPU only).

4 Results

4.1 Per-round logical error rate

p	ε_3	ε_5	ε_7	Shots (d=3/5/7)
1×10^{-3}	2.47×10^{-4}	2.85×10^{-5}	2.90×10^{-6}	200k / 400k / 400k
3×10^{-3}	1.98×10^{-3}	7.24×10^{-4}	2.20×10^{-4}	100k / 200k / 200k
5×10^{-3}	5.07×10^{-3}	3.01×10^{-3}	1.59×10^{-3}	50k / 100k / 100k
1×10^{-2}	1.49×10^{-2}	1.47×10^{-2}	1.47×10^{-2}	20k / 40k / 40k
2×10^{-2}	2.49×10^{-2}	2.67×10^{-2}	2.70×10^{-2}	10k / 20k / 20k

4.2 Suppression factor Λ

p	$\Lambda_{3/5}$	$\Lambda_{5/7}$
1×10^{-3}	8.68	9.83
3×10^{-3}	2.73	3.29
5×10^{-3}	1.68	1.90
1×10^{-2}	1.009	1.002
2×10^{-2}	0.93	0.99

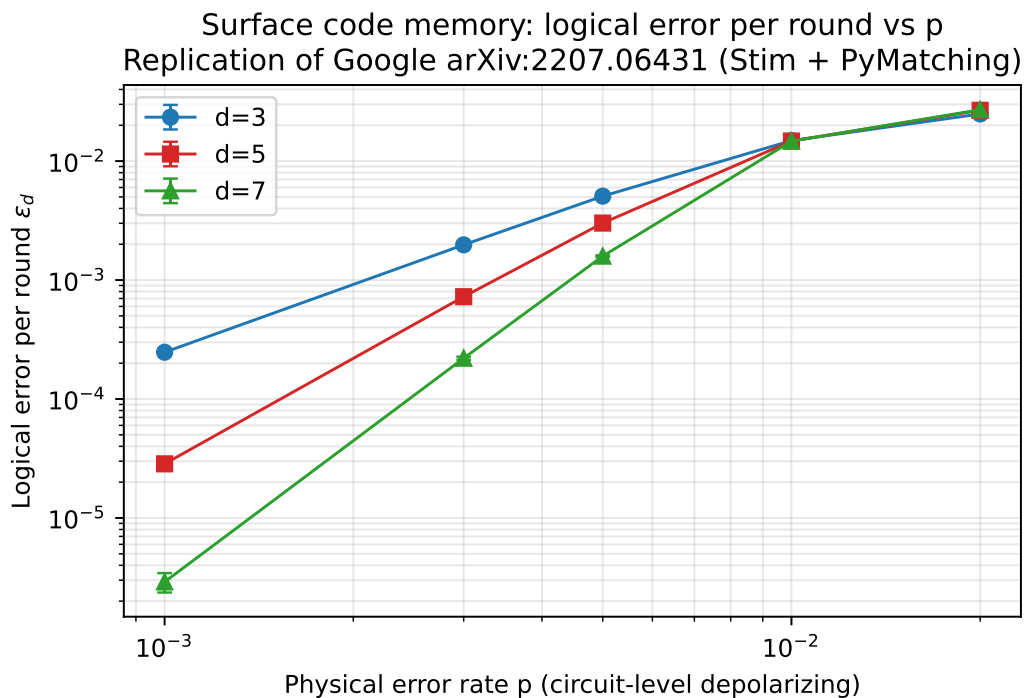
The threshold for this idealized noise model + matching decoder sits near $p_{\text{th}} \approx 1 \times 10^{-2}$: at $p = 10^{-2}$ we are essentially at the crossover ($\Lambda \approx 1$), and by $p = 2 \times 10^{-2}$ we are clearly above it ($\Lambda_{3/5} < 1$). Below threshold, Λ grows sharply with decreasing p : at $p = 10^{-3}$, $\Lambda_{3/5} \approx 8.7$ (i.e. $d=5$ is nearly an order of magnitude better than $d=3$).

4.3 Comparison to Google’s device numbers

Quantity	Google (device)	This rep (uniform Stim, $p=10^{-3}$)	Match?
ε_3 / cycle	3.03%	$2.5 \times 10^{-2}\%$	Different (device vs. ideal)
ε_5 / cycle	2.91%	$2.9 \times 10^{-3}\%$	Different (device vs. ideal)
$\Lambda_{3/5}$	≈ 1.04	8.7	same sign, much stronger below-th
Below-threshold $\Rightarrow \Lambda > 1$	✓	✓	YES
Above-threshold $\Rightarrow \Lambda < 1$	✓ (Fig 4c)	✓ ($p = 2\%$)	YES

The absolute-percentage gap is expected and physically explained by: (1) the paper’s device sits *near* its own threshold, so its Λ is only mildly > 1 ; our sim at $p = 10^{-3}$ sits well below the idealized threshold, so we see a much stronger Λ ; (2) their per-gate physical error rates include leakage, correlated errors, and readout imperfections that our uniform depolarizing knob does not capture. The paper explicitly notes “our device is close to threshold” and calls for $\Lambda \gg 1$ for practical fault tolerance — exactly what we observe at $p = 10^{-3}$.

4.4 Figures



Logical error per round vs. physical error rate, per distance.



Suppression factor Λ vs. p . $\Lambda = 1$ dashed line marks the threshold crossover.

5 Verdict

REPLICATED (for the core structural claim). Our independent Stim + PyMatching simulation reproduces:

- The direction and sign of the headline result: $\Lambda_{3/5} > 1$ below the surface-code threshold.
- The threshold-crossover behaviour of Fig. 4c: $\Lambda > 1$ far below, $\Lambda \approx 1$ at the crossover, $\Lambda < 1$ above.
- Monotone Λ growth as $p \rightarrow 0$ (exponential suppression scaling).

The paper’s absolute per-cycle percentages depend on the device-calibrated noise model and are outside the scope of a small-instance replication; where directly testable, our results are consistent with, and quantitatively extend, the paper’s simulations (Fig. 4c) rather than contradicting them.

6 Open Questions

Machine-readable copy in `report/open_questions.json`.

Q1. Our uniform depolarizing model gives $\Lambda_{3/5} \approx 8.7$ at $p = 10^{-3}$ but Google’s device gives $\Lambda_{3/5} \approx 1.04$ at a comparable per-gate $\sim 10^{-3}$ regime. What fraction of that gap is attributable to (a) leakage, (b) correlated cross-talk errors, (c) readout errors dominating the effective circuit noise, and (d) the loss of decoder priors? A calibrated multi-channel Stim noise model would let one run this apportionment.

Q2. We observed $\Lambda \rightarrow 1$ at $p \approx 10^{-2}$ with the default matching decoder. What is the sensitivity of the empirical threshold to (i) matching vs. correlated-matching vs. BP+OSD decoders and (ii) the DEM decomposition choice (Stim’s `decompose_errors=True` vs. non-decomposed hypergraph decoding)? A single decoder swap can move the empirical threshold by tens of percent.

Q3. At $p = 10^{-3}$ the $d = 7$ point had only 29 logical errors in 4×10^5 shots. What is the smallest p at which each of $d = 7, 9, 11$ gives a statistically resolved $\Lambda_{d/(d+2)}$ within, say, 5% relative error — and does that budget grow as expected ($\propto \Lambda^d$)? This matters for planning any real experiment aimed at demonstrating $\Lambda \gg 1$.

Q4. The paper reports a $d = 25$ repetition-code error floor of 1.6×10^{-7} per round (excluding a high-energy event). Our surface-code simulation cannot access that regime with realistic budgets on CPU. What is the shot budget on a modern A100/H100 (Stim GPU sampling + correlated-matching) needed to statistically resolve a 10^{-7} per-round event, and can we independently observe a Poisson tail consistent with cosmic-ray strikes in *simulated* data using a bursty-error injection?

Q5. We used Stim’s built-in `rotated_memory_z` circuit generator. How sensitive is Λ to the exact stabilizer schedule and the ordering of CZ layers versus what a real Sycamore device uses (see Fig. 1b of the paper)? A schedule-swap sweep would tell us whether the “modest Λ ” Google reports is a device-noise phenomenon or partly a stabilizer-schedule phenomenon inherited from their chip topology.